

Excellent choice. Since you've already completed:

-  Text Preprocessing
-  N-Grams
-  Bag of Words
-  TF
-  IDF
-  TF-IDF
-  Cosine Similarity

an **FAQ Search Engine** is the perfect first NLP project because it uses almost everything you've learned. It also looks good in a portfolio and is commonly used in real applications.

Project 1: FAQ Search Engine

GOAL

A user asks a question in natural language.

Example:

 "How can I reset my password?"

Your application searches all FAQs and returns the most similar question with its answer.

Example Output

```
User:
I forgot my password

Best Match:
How do I reset my password?

Similarity: 0.91

Answer:
Go to Login → Click "Forgot Password" → Follow the email instructions.
```

Real-world Example

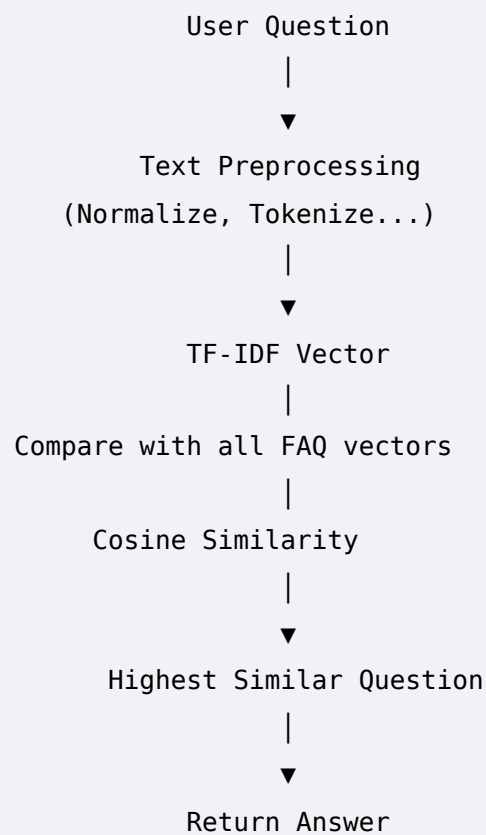
Companies use this everywhere.

- Microsoft Support

- Amazon Help Center
- Banking FAQ
- Hospital FAQ
- HR Chatbot
- University Help Desk

Instead of manually reading hundreds of FAQs, users type a question.

Architecture



Folder Structure

FAQSearchEngine

```
|
|— Models
|   FAQ.cs
|
|— Data
|   faq.json
|
```

```
├─ Services
|   TextPreprocessingService.cs
|   TFService.cs
|   IDFService.cs
|   TFIDFService.cs
|   CosineSimilarityService.cs
|   SearchService.cs
|
└─ Program.cs
```

This keeps the project clean and modular.

Step 1 — Create FAQ Model

```
public class FAQ
{
    public int Id { get; set; }

    public string Question { get; set; }

    public string Answer { get; set; }
}
```

Very simple.

Step 2 — FAQ JSON

Example

```
[
  {
    "id":1,
    "question":"How do I reset my password?",
    "answer":"Click Forgot Password."
  },
  {
    "id":2,
    "question":"How can I change my email?",
    "answer":"Go to Profile Settings."
  },
  {
    "id":3,
```

```
    "question": "How do I update my phone number?",
    "answer": "Go to Personal Information."
  },
  {
    "id": 4,
    "question": "How can I contact support?",
    "answer": "Email support@company.com."
  },
  {
    "id": 5,
    "question": "Where can I download my invoice?",
    "answer": "Go to Billing."
  }
]
```

Later you can expand this to 500–1000 FAQs.

Step 3 — Load FAQs

faq.json

↓

List<FAQ>

↓

Memory

No database is needed for this project.

Step 4 — Preprocess Every FAQ

For every question:

Original

How do I reset my password?

↓

Normalize

how do i reset my password

↓

Remove punctuation

how do i reset my password

↓

Tokenize

how

do

i

reset

my

password

↓

Remove stop words

reset

password

↓

Stem/Lemmatize

reset

password

Store the processed tokens.

Step 5 — Build Vocabulary

Collect all unique words from every FAQ.

Example

```
reset
password
email
phone
invoice
billing
support
contact
profile
change
```

This vocabulary is used for TF-IDF vectors.

Step 6 — Calculate TF-IDF

For every FAQ

```
FAQ 1
```

```
reset
password
```

```
↓
```

```
TF
```

```
↓
```

```
IDF
```

```
↓
```

```
TF-IDF Vector
```

Store every vector.

Example

```
FAQ1
```

```
[0.72,
0.64,
```

```
0,  
0,  
0,  
0,  
...]
```

FAQ2

```
[0,  
0,  
0.58,  
0,  
0.81,  
...]
```

This preprocessing happens once when the application starts.

Step 7 — User Searches

Suppose the user types

Forgot password

Process it exactly the same way.

Normalize

↓

Tokenize

↓

Remove Stop Words

↓

Lemmatize

↓

TF-IDF Vector

Consistency between stored FAQs and queries is essential.

Step 8 — Compare with Every FAQ

Compute cosine similarity between the user's vector and each FAQ vector.

User
↓
FAQ1
0.92
FAQ2
0.15
FAQ3
0.05
FAQ4
0.18
FAQ5
0.02

Step 9 — Return Best Match

Pick the highest similarity score.

Question:
Forgot password
↓
Best FAQ

How do I reset my password?

↓

Answer

Click Forgot Password.

Example Flow

FAQs

Q1

How do I reset my password?

Q2

How can I change my email?

Q3

Where can I download invoices?

User

I forgot password

↓

Preprocessing

↓

TF-IDF

↓

Cosine Similarity

Q1 0.93

Q2 0.21

Q3 0.05

↓

Return

[Click Forgot Password.](#)

Services You Will Build

Service	Responsibility
TextPreprocessingService	Normalize, tokenize, remove stop words, stem/lemmatize
VocabularyService	Build vocabulary from all FAQs
TFService	Calculate term frequency

IDFService	Calculate inverse document frequency
TFIDFService	Generate TF-IDF vectors
CosineSimilarityService	Compare vectors
SearchService	Execute search and return the best match
JsonService	Load FAQs from JSON

Suggested Implementation Order

1.  Create the project
2.  Create `FAQ` model
3.  Create `faq.json`
4.  Load FAQs into memory
5.  Preprocess all questions
6.  Build the vocabulary
7.  Calculate IDF for the corpus
8.  Generate and cache TF-IDF vectors for all FAQs
9.  Process the user's query

10.  Generate the query's TF-IDF vector
 11.  Calculate cosine similarity with each FAQ
 12.  Sort by similarity and return the best match (or top 3 matches)
-

What You'll Learn

By completing this project, you'll reinforce:

- Text preprocessing
- Vocabulary creation
- TF calculation
- IDF calculation
- TF-IDF vectorization
- Cosine similarity
- Information retrieval basics
- Search ranking
- Clean service-oriented architecture in C#

This project also prepares you for more advanced topics. The next projects—**Resume Search**, **Similar Document Finder**, **Semantic Search**, and eventually **RAG (Retrieval-Augmented Generation)**—all follow a similar retrieval pipeline, but replace TF-IDF with embeddings and vector search. Starting with this FAQ Search Engine gives you a solid understanding of the underlying concepts before moving to modern NLP techniques.